第四周周报（3.26-4.1）

第 2 组 OS 课设周报-第 4 周-杜昊阳-富思涵-赫金科-康忆文-卢彦芸-陆昱衡-米梓润-吴安然

本周开始周报主要描述上周计划的执行情况及下周工作计划等内容

一、本周计划执行情况

1. 内存管理：杜昊阳，赫金科

- 新增内存映射可视化工具 (apps/memmap/memmap.c:1-129)
  - 创建图形界面应用程序，实时显示 128MB 物理内存使用状态
  - 使用 256×128 像素网格，每个像素代表 4KB 内存页
  - 黑色表示已分配内存，白色表示空闲内存
  - 显示已分配内存、空闲内存、空闲块数量等统计信息
- 新增系统调用 API (apps/apilib/api040.nas:1-18)
  - API 36 号：api_getmemmap - 获取内存映射信息
  - 支持返回最多 4090 个空闲内存块条目
  - 返回空闲字节总数
- 内核层支持 (kernel/console.c:728-746)
  - 在中断处理中添加 API 36 号处理逻辑
  - 将内存管理器的空闲块信息复制到用户缓冲区
- 数据结构定义 (apps/apilib.h:14-19)
  - 新增 MEMMAP_ENTRY 结构体，包含地址和大小字段

2. 中断：吴安然

本周任务目标： 阅读内核中与定时器、定时器中断相关的代码与资料，理清定时器管理数据结构与工作流程，并在指导教师要求范围内初步确定「扩展定时器与测试」的技术路线与文档提纲。

- 阅读范围梳理

根据项目《kernel 各模块作用》说明，将本人负责的「中断 / 定时器」相关代码锁定在 `kernel/timer.c`、`kernel/int.c`（PIC 初始化）、`kernel/dsctbl.c`（IDT 中 IRQ0 向量）、`kernel/naskfunc.nas`（定时器中断入口）及 `kernel/bootpack.h` 中 `TIMER`、`TIMERCTL` 等类型定义。

- 建立阅读顺序

采用「先果后因」的方式：优先阅读 `inthandler20`（IRQ0 中断服务程序），理解全局节拍 `timerctl.count` 自增、与 `timerctl.next` 配合的早期返回优化、按链表处理到期定时器、向 FIFO 投递数据及与 `task_timer` 相关的任务切换；再阅读 `init_pit` 与 PIT 端口配置，最后阅读 `timer_alloc`、`timer_settime`、`timer_cancel` 等与链表维护相关的接口。

- 原理与结构归纳

归纳出本系统以 8254 PIT 通道 0 为单一硬件节拍源，经 8259 PIC 的 IRQ0 进入内核；软件定时器以 `TIMERCTL` + 按到期时间排序的链表 管理，池中最多 `MAX_TIMER` 个 `TIMER` 结构。明确课程任务「设定更多的定时器」在本架构下主要指在同一套 PIT+IRQ0 下增加并发软件定时器或调整池规模，并配套测试「管理结构与管理方法」。

- 方案初定

初步确定后续实验将围绕：（1）在现有 API 下增加并发/压力场景或适度扩展定时器数量；（2）设计针对链表插入顺序、到期顺序、`timer_cancel` 与中断并发等方面的测试用例；（3）记录测试环境、步骤与现象，形成可复现的测试说明。具体指标与是否涉及 PIT 多通道等以第二周与指导教师确认为准。

3. 进程调度：富思涵，卢彦芸，康忆文

- 内核调度逻辑分析：梳理了 `task_now()` 与 `task_switch()` 的交互流程。分析了当前系统的优先级层级实现方式（Level 0-9），确认任务在不同层级间移动的触发条件。计划引入更灵活的动态优先级调整算法，以防止低优先级任务出现“饥饿”现象。

- 竞争条件与内核互斥研究：在现有内核中发现，由于中断（如定时器、键盘）会打断当前任务，直接访问全局变量（如 `memman`）存在竞争风险。 研究 `io_cli()` 和 `io_sti()` 在保护临界区中的作用，并计划实现基于自旋锁（Spinlock）或信号量（Semaphore）的内核级互斥机制。

- 同步场景设计：针对生产者-消费者模型，设计了基于内核 FIFO 缓冲区的初步方案。构思如何在用户态通过系统调用（INT 0x40）申请、释放信号量，从而解决多个用户进程访问共享显存或内存块时的同步问题。

- 系统调用接口规划：确定了通过修改 `TASK` 结构体增加 `status`（睡眠/就绪）标志位，实现进程阻塞挂起。

    - `api_sem_init` (edx=80)：初始化信号量。

- `api_sem_p` (edx=81)：P 操作（申请资源，阻塞）。
- `api_sem_v` (edx=82)：V 操作（释放资源，唤醒）。

- 方案初定为实验路径：(1) 实现内核态的信号量管理逻辑；(2) 扩展 `task_sleep` 和 `task_run` 以支持基于等待队列的阻塞唤醒；(3) 编写用户态测试程序（如 `philosopher.c`）演示三个及以上进程在竞争资源时的死锁预防与同步。

4. 文件系统：米梓润，陆昱衡

- 内核侧 memfs 模块

1. 新增内核模块：

> kernel/memfs.c、kernel/memfs.h

2. 以内存分配器（memman_alloc_4k）

   获取一片连续内存作为“memdisk”。

3. 以簇$cluster$为分配单位，FAT（u16）维护簇链：

   0=空闲、0xFFFF=链尾、

   其他值指向下一簇。

4. 固定目录项数组（如 256 项），通过$parent$ 字段实现层级目录；路径采用 `/` 分隔。

5. 提供基础操作：格式化、建目录、建文件、读/写（支持 offset）、复制、删除、列目录。

- 在 $kernel/console.c$ 的 $hrbapi()$分发$edx = 70..77$中新增：

70：format（指定 memdisk 大小，KB）
71：mkdir
72：create
73：write（path + offset + buf + len）
74：read（path + offset + buf + len）
75：copy（src, dst）
76：delete
77：list（path + outbuf + outlen）

1. 应用侧 apilib 封装$apps/apilib/api070.nastoapi077.nas$
按工程惯例：寄存器传参 + $int0x40$

2. 修改 $apps/apilib/Makefile$，将 api070.obj~api077.obj 打入 apilib.lib。

3. 修改 $apps/apilib.h$，补充应用侧函数声明，方便 C 程序调用。

- 控制台演示命令
- memfs：显示 memfs 命令帮助
- mformat <kb>
- mmkdir <path>
- mcreate <path>
- mwrite <path> <off> <text>off`为字节偏移
- mread <path> <off> <len>
- mcopy <src> <dst>
- mls <path>
- mrm <path>

二、下周工作计划

1. 内存管理：杜昊阳，赫金科

- 实现五种内存分配算法
- 新增算法查询功能
- 新增系统调用 API 37 号：api_getmemalgo - 获取当前内存分配算法

2. 中断：吴安然

- 与指导教师对齐「更多定时器」的范围与验收标准，并留下邮件或聊天记录摘要。
- 完成《定时器子系统测试方案》初稿，写清目的、环境、用例、预期与记录方式。
- 精读 `timer_settime`、`timer_cancel` 与 `inthandler20` 的衔接，整理关中断与链表一致性的笔记或简图。
- 在 `mtask.c` 中标注与 `task_timer`、`task_switch` 相关的接口，形成清单以免改动破坏调度。
- 与内存/调度方向同学同步分支合并计划，并写简短备忘附在周报。

3. 进程调度：富思涵，卢彦芸，康忆文

- 内核同步原语的底层实现，信号量结构体定义： 在 `bootpack.h` 中新增 `struct SEMAPHORE`，包含信号量计数器 `count`、等待队列指针 `struct TASK *wait_tasks[MAX_TASKS]` 以及队列长度，用于管理因资源竞争而阻塞的任务。
- 阻塞与唤醒机制： 修改 `mtask.c`。实现当信号量 $count \le 0$ 时，调用

`task_sleep` 将当前进程从运行层级（Level）中移除；并在 V 操作（释放资源）时，通过 `task_run` 将等待队列首位的进程重新唤醒并加入就绪队列。

- 原子操作保证： 确保 P/V 操作在执行期间通过 `io_cli()` 屏蔽中断，防止在修改信号量状态时发生任务切换，从而规避内核态的竞争条件。
- 中断处理分发： 在 `kernel/console.c` 的 `hrb_api` 函数中新增 80-83 号功能号的分发逻辑：
  - API 80 (sem_init): 申请并初始化一个内核信号量。
  - API 81 (sem_p): 封装内核 P 操作，供用户态程序调用以申请资源。
  - API 82 (sem_v): 封装内核 V 操作，供用户态程序调用以释放资源。
  - API 83 (sem_free): 销毁信号量并释放相关内核内存。
- 寄存器传参规范： 参照 `apps/apilib` 惯例，使用 `EBX` 传递信号量 ID，`ECX` 传递初始值，在 `naskfunc.nas` 中完成对应的汇编跳转封装。

4. 文件系统：米梓润，陆昱衡

- 测试程序与镜像打包

1. 新增测试应用目录：apps/memfstest（用于快速验证 memfs 全链路）
2. 关键问题与修复：
3. 链接警告 `can't link __main`：按工程 app 入口约定调整为 `_main()`。
4. 将 `mfstest.hel` 加入顶层构建与镜像打包流程（确保进入 OS 后可直接运行）。